Applied Soft Computing Journal 102 (2021) 107113

Contents lists available at ScienceDirect

Applied Soft Computing Journal

journal homepage: www.elsevier.com/locate/asoc

Distributed Grey Wolf Optimizer for scheduling of workflow applications in cloud environments

Bilal H. Abed-alguni *, Noor Aldeen Alawad

Department of Computer Sciences, Yarmouk University, Irbid, Jordan

ARTICLE INFO

Article history:
Received 6 December 2019
Received in revised form 9 November 2020
Accepted 12 January 2021
Available online 21 January 2021

Keywords:
Distributed Grey Wolf Optimizer
Cloud computing
Optimization
Workflow
Scheduling
Load balancing

ABSTRACT

Optimal scheduling of workflows in cloud computing environments is an essential element to maximize the utilization of Virtual Machines (VMs). In practice, scheduling of dependent tasks in a workflow requires distributing the tasks to the available VMs on the cloud. This paper introduces a discrete variation of the Distributed Grey Wolf Optimizer (DGWO) for scheduling dependent tasks to VMs. The scheduling process in DGWO is modeled as a minimization problem for two objectives: computation and data transmission costs. DGWO uses the largest order value (LOV) method to convert the continuous candidate solutions produced by DGWO to discrete candidate solutions. DGWO was experimentally tested and compared to well-known optimization-based scheduling algorithms (Particle Swarm Optimization (PSO), Grey Wolf Optimizer). The experimental results suggest that DGWO distributes tasks to VMs faster than the other tested algorithms. Besides, DGWO was compared to PSO and Binary PSO (BPSO) using WorkflowSim and scientific workflows of different sizes. The obtained simulation results suggest that DGWO provides the best makespan compared to the other algorithms.

1. Introduction

A workflow application is a commonly used term to describe applications that comprise dependent tasks (i.e., applications with data or control flow dependencies [1,2]). Cloud computing is a term used to describe a network of remote servers on the Internet that provides several services such as storage management and data processing services [3–5]. It provides Virtual Machines (VMs) or compute resources to execute workflow applications (e.g., bioinformatics, astronomy and physics applications [6]). A key factor to the success of data processing service in cloud environments is the scheduling techniques that schedule workflow applications on cloud environment such that the cost of using compute resources is minimized. However, the task scheduling problem in cloud computing environments is an $\mathcal{NP}$-hard problem [7]. Therefore, several researchers have attempted in the recent years to find solutions for the scheduling problem using optimization-based scheduling algorithms (e.g., Particle Swarm Optimization (PSO) [8], PSO with load balancing mutation [9], Pareto-based Grey Wolf Optimizer (PGWO) [10], Multi-objective ant colony system (MOACS) [11], hybrid Gravitational Search Algorithm (GSA) [12], Genetic algorithm (GA) using an adaptive penalty function [13] and hybrid Bat and Binary Bat algorithm (BBA) [14]). The optimization algorithms which are the bases of the optimization-based scheduling algorithms may easily get trapped in local optima earlier than expected because of some limitations in their exploration methods [15–19]. Besides, the performance of the optimization algorithms degrades when dealing with medium and high dimensional optimization problems. It is also important to note that the hybrid scheduling algorithms (e.g., hybrid GSA, hybrid BBA) require normally more execution time than the traditional scheduling optimization algorithms (e.g., GSA, BA). This is because the hybrid optimization algorithms integrate in their optimization loops one or more local or global search methods. Therefore, the length of an iteration of a hybrid algorithm is the sum of the time required for the optimization operators to complete and the time required for the integrated search method to finish execution. Besides, the execution time of an iteration varies from one iteration to another depending on the complexity of the integrated search method inside the hybrid algorithm. This means that the average length of an iteration in the hybrid optimization algorithms is longer than the average length of an iteration in the traditional optimization algorithms [16–18].

The Distributed Grey Wolf Optimizer (DGWO) is a parallel version of the Grey Wolf Optimizer (GWO) algorithm [18]. This means that the optimization process of DGWO can be performed in parallel machines. There are two main reasons that make DGWO an interesting choice for solving optimization problems.

* Corresponding author.
E-mail addresses: Bilal.h@yu.edu.jo (B.H. Abed-alguni), nooraldeen@yu.edu.jo (N.A. Alawad).

https://doi.org/10.1016/j.asoc.2021.107113
1568-4946/

First, the DGWO has faster convergence rate than popular optimization algorithms such as the Grey Wolf Optimizer, Cuckoo search [20–22], memory-based hybrid Dragonfly algorithm [23] and Fireworks algorithm with differential mutation [24]. Second, DGWO has one parameter (the vector $\overrightarrow{\boldsymbol{a}}$ (Section 3.1)) which does not require fine tuning.

In this paper, we propose an optimization-based scheduling algorithm for workflow applications based on the DGWO algorithm. We model the task scheduling problem of workflow applications in the proposed algorithm as a minimization problem (i.e., provide a mapping of dependent tasks of a workflow application to compute resources on a cloud computing environment such that the total execution cost (i.e., computation and data transmission costs) of the application is minimized). Finally, the candidate solutions in the scheduling problems can be generated in DGWO using the largest order value (LOV) method [25] as described in Section 4.1. We experimentally evaluated DGWO against well-known optimization-based scheduling algorithms such as Particle Swarm Optimization (PSO) [8] and GWO [26] using two types of workflows: balanced and imbalanced workflows. We noticed that the overall experimental results on balanced workflows indicate that DGWO outperforms the other algorithms when applied to scheduling problems with various data sizes. Moreover, we noticed that the overall experimental results conducted on imbalanced workflows indicate more clearly that DGWO performs better than the other algorithms.

To sum up, the main contributions of this paper are summarized as follows:

1. We introduce a discrete variation of DGWO (Algorithm 4) that can be used to solve various types of scheduling problems.
2. We modify the dynamic scheduling algorithm proposed in [8,10] in order to make it suitable to DGWO. Like the original dynamic scheduling algorithm, the modified algorithm can deal with both balanced and imbalanced workflow applications. Algorithm 3 aims to minimize the computation and data transmission costs.
3. We use WorkflowSim and real scientific workflows to evaluate the performance of DGWO against the simulation results reported in [27] for two algorithms: PSO and Binary PSO (BPSO). The experimental results suggest that DGWO provides the lowest makespans for different real scientific workflows with different sizes. Besides, it indicates that the performance of DGWO improves with the increase of size of workflow compared to BPSO and PSO.
4. We also conduct experiments to evaluate and compare DGWO to two popular scheduling algorithms, PSO and GWO, using balanced and imbalanced workflows. The experimental results show that DGWO is the fastest converging algorithm.

The rest of the paper is organized as follows: Section 2 provides a review of recent methods that attempt to solve the task scheduling problem. Section 3 provides background discussions about the Grey Wolf Optimizer algorithm, the Distributed Grey Wolf Optimizer and the task scheduling problem in workflow applications. Section 4 discusses the proposed scheduling algorithms in details. Section 5 presents the experimental results. Finally, Section 6 presents the conclusions of this paper and discusses some future work directions.

2. Recent work

The task scheduling problem in cloud computing environments is an $\mathcal{NP}$-hard problem [7]. In the recent years, several researchers have attempted to find solutions for the scheduling problem using optimization-based scheduling algorithms [8–14, 28–34]. This section provides an overview of recently proposed optimization-based scheduling algorithms.

Pandey et al. [8] proposed Particle Swarm Optimization-based Heuristic, one of the first optimization-based scheduling algorithms, based on the Particle Swarm Optimization (PSO) algorithm. In this work, PSO is used to schedule dependent tasks to cloud resources based on two objectives: computation and data transmission costs. The experimental results in [8] suggest that PSO converges to sub-optimal solutions three times faster than the Best-Resource-Selection algorithm. Awad et al. [9] incorporated a new mutation technique called LBM (i.e., a load balancing technique for minimizing makespan, execution time, round trip time and transmission cost) into the PSO algorithm to produce a new algorithm called the LBMPSO algorithm. LBMPSO is a four-objective optimization algorithm compared to the particle Swarm Optimization-based Heuristic algorithm which is a two-objective optimization algorithm. The experimental results in [9] suggest that LBMPSO performs better than PSO and the random resource-allocation algorithm in minimizing the makespan, execution time, round trip time and transmission cost. However, PSO and its variations may easily get trapped in local optima and have in general slow convergence rate [35].

Pareto-based Grey Wolf Optimizer (PGWO) is an algorithm for scheduling workflow applications in cloud computing environments [10]. It uses the smallest position value rule to determine the fitness value of candidate solutions in GWO. PGWO is a multi-objective optimization algorithm that aims to minimize the execution time of dependent tasks, minimize the cost of using compute resources and maximize the overall throughput of the compute resources. The experimental results in [10] suggest that PGWO is a better scheduling algorithm than the improved strength Pareto evolutionary algorithm (SPEA2) [36]. A problem with the GWO and PGWO algorithms is that they may converge earlier than anticipated to sub-optimal solutions.

Multi-objective ant colony system (MOACS) is a discrete optimization algorithm that uses two populations to solve workflow scheduling problems with multiple objectives [11]. MOACS creates a population for each objective: a population to minimize the execution time and another one to minimize the execution cost. Another interesting feature of MOACS is that it adopts a new pheromone update rule. This rule uses a set of non-dominated candidate solutions from a global archive to amend the diversity of its two populations and guarantee the efficiency of the search process. MOACS has been experimentally proven in [11] to be more efficient in solving workflow scheduling problems than ECMSMOO, MOHEFT, NSGA-II, EMS-C and MODE. It is important to note that the sequence of random decisions are not dependent on the Multi-objective ant colony algorithm and the single-objective ant colony algorithm [37]. In addition, the ant colony algorithms may prematurely converge to inadequate solutions when applied to high dimensional problems.

The Gravitational Search Algorithm (GSA) and the Heterogeneous Earliest Finish Time (HEFT) algorithm were incorporated in one algorithm called HGSA to provide efficient mapping of tasks to resources in workflow applications [12]. HGSA is a bi-objective algorithm that aims to reduce the makespan and execution time of a scheduling problem. The experimental and statistical results in [12] indicate that HGSA provides better mapping of tasks than GSA and the Genetic algorithm (GA). A disadvantage of HGSA relies on fact that it is a hybridization of two algorithms which means that the average length of an iteration in HGSA is longer than the average length of an iteration in GSA or HEFT.

Liu et al. [13] suggested a modified GA algorithm using an adaptive penalty function for the strict constraints. The modified GA algorithm uses a co-evolution approach to dynamically

modify the crossover and mutation probabilities to accelerate the convergence and enhance the diversity of the population. The algorithm was compared to PSO, HEFT and the basic GA algorithm in the WorkflowSim simulator to evaluate their efficiency in solving workflow scheduling problems. The simulations results provide an evidence of the efficiency of the modified GA algorithm [13].

The Bat algorithm (BA) is a population-based optimization algorithm that is inspired from the forging behavior of microbats. It is mainly used to solve continuous optimization problems. On the other hand, the Binary Bat algorithm (BBA) is a discrete variation of BA. Raghavan et al. [14] has combined BA and BBA in one algorithm to schedule workflow applications in cloud environments. Another variation of BA, Wavelet Perturbation-based Bat algorithm (WPBA), has been used as the bases of a new scheduling algorithm [38]. The WPBA scheduling algorithm utilizes wavelet perturbation to enhance the performance of Bat algorithm then uses population-entropy-guided substitution to improve the convergence speed and the population of the algorithm. The simulation results in [38] indicate that the WPBA scheduling algorithm is better than BA when they are implemented as an optimization-based scheduling algorithms. A possible problem with the BA algorithm and its variations is that they may easily fall into local optima because of some limitations in their exploration abilities [39].

OTB-CSO is a hybrid optimization algorithm that incorporates the Taguchi orthogonal approach into the tracing mode of Cat Swarm Optimization algorithm [40]. It aims to minimize the execution time of workflow applications in cloud computing environments. Although OTB-CSO is an efficient algorithm, it requires longer running time to produce good solutions compared to some traditional optimization algorithms such as GWO and BA algorithms.

The oppositional Cuckoo Search algorithm (OCSA) [41] is an optimization algorithm that combines the opposition-based learning approach into the CS algorithm to provide a solution for the task scheduling problem in cloud computing. The opposition-based learning in OCSA is based on the opposite relationships among the decision variables in candidate solutions (e.g., lower bound vs upper bound of a decision variable). The simulation results in [41] suggest that OCSA is a better scheduling algorithm compared to PSO and GA algorithms. However, OCSA does not guarantee that all the candidate solutions will not fall in one area of the search space when generating or updating the population of candidate solutions.

Abdullahi et al. [30] proposed the chaotic symbiotic organisms search (CMSOS) algorithm in order to provide solutions for multi-objective task scheduling problem on cloud computing environment. In CMSOS, the chaotic optimization approach is used to generate the population of candidate solutions and the chaotic sequence technique is used to ensure the diversity of the population. In addition, the chaotic local search technique in CMSOS is used to minimize the possibility of entrapment in local optima. CMSOS has proved its efficiency in standard and synthetic workload instances. However, the optimization operators in CMSOS do not guarantee a good balance between exploration and exploitation of the search space [42].

The utilization of VMs is a challenge in cloud computing environment. The intelligent Ant colony algorithm (ACO) (i.e., a variation of the ant colony algorithm for scheduling workflows in cloud computing environment [43]) allocates underutilized VMs using Pareto distribution. The ACO algorithm aims to maximize the usage of VMs while minimizing both the execution time and execution cost of the workflows. It is important to note that the optimization process of ACO depends on many optimization operators and the probabilistic distribution of the candidate solutions in ACO may change with high probability at each iteration of its optimization process [29].

As previously discussed, optimization-based scheduling algorithms have been repetitively used in the literature [8–14,28–34,40,41] to solve the scheduling problem of workflows in cloud computing environments because of their known efficiency in solving $\mathcal{NP}$-hard problems. However, premature convergence (i.e., a term used to describe the lost of diversity in the population of an optimization algorithm at the beginning of its simulation process) is one of the main problems of the optimization algorithms used in the optimization-based scheduling algorithms due to some shortcomings in their exploration methods [8–11]. In addition, the performance of some optimization algorithms depends heavily on the size of the scheduling problem. In fact, the performance of these algorithms is proportional to the size of the scheduling problem [12–14].

The Omega system is known as Google's next-generation cluster control system [3]. It is based on the concepts of parallelism, shared state, and optimistic concurrency management. The performance of Omega was evaluated using light-weighted simulations with artificial workloads and heavy-weighted simulations with real workloads at Google. The experimental results in [3] indicate that the optimistic concurrency over shared state is a feasible, efficient approach to cluster scheduling. Mesos [44] is a another platform for sharing commodity clusters among various computing frameworks for clusters (e.g., Hadoop and MPI.). Mesos includes "resource offers", which is a distributed two-level scheduling mechanism. It determines the number of resources to offer each framework, while the frameworks determine the resources and computations to be executed on them. The simulation results in [44] suggest that existing frameworks can efficiently share resources using Mesos.

We use the DGWO in the current paper as the bases for a new scheduling algorithm to deal with the limitations and disadvantages of many of the current optimization-based scheduling algorithms. This is because DGWO has faster convergence rate than well-known algorithms such as the Grey Wolf Optimizer, Cuckoo search [20–22], memory-based hybrid Dragonfly algorithm [23] and Fireworks algorithm with differential mutation [24]. Also, DGWO has one parameter (the vector $\overrightarrow{\boldsymbol{a}}$ (Section 3.1)) which does not require fine tuning. In addition, the candidate solutions in the task scheduling problem can be easily generated in DGWO using discrete numbers as described in the next section.

3. Preliminaries

This section provides a summary of some of the underlying concepts of the Grey Wolf Optimizer algorithm (Section 3.1) and the Distributed Grey Wolf Optimizer algorithm (Section 3.2). The final part of the section (Section 3.3) provides a description about the mathematical formulation of the task scheduling problem in cloud computing environments.

3.1. Grey Wolf Optimizer

The Grey Wolf Optimizer (GWO) is a population-based optimization algorithm that is capable of solving various types of optimization problems: discrete, continuous, constrained and non-constrained problems [18,26]. The optimization process of GWO simulates the cooperative hunting strategy and hierarchical leadership structure of the Grey Wolves. A pack of wolves in the nature has a social structure that reflects the strength of each wolf in the wolf pack. A couple of wolves, called alpha (α) male and alpha female, controls the pack of wolves and organizes the hunting of prey to feed the members of the pack. The beta (β) and delta (δ) wolves represent the second level of leadership in

the pack. Their main duty is to assist the α wolves to enforce their authority on the rest of wolf pack (Omega (ω) wolves). The cooperative hunting strategy of the Grey Wolves is an interesting behavior that represents the core idea of the optimization process of GWO. This strategy comprises three actions: tracking and chasing prey, pursuing and encircling prey and attacking prey.

Algorithm 1 The Grey Wolf Optimizer (GWO).

1: Initialize the population of n candidate solutions $\vec{X}(i = 1, 2, , n)$
2: Initialize $\vec{A}$, $\vec{a}$ and $\vec{C}$.
3: Calculate the fitness value of each candidate solution $\vec{X}_i$ using its fitness function $f(\vec{X}_i)$
4: $\vec{X}_\alpha$= the best candidate solution
5: $\vec{X}_\beta$= the second best candidate solution
6: $\vec{X}_\delta$= the third best candidate solution
7: **while** ($t <$*MaxItr*) **do**
8: **for** each candidate solution **do**
9: Update the values of the current candidate solution using Eq. (11).
10: **end for**
11: Update $\vec{A}$, $\vec{a}$ and $\vec{C}$.
12: Calculate the fitness value of each candidate solution
13: Update $\vec{X}_\alpha$, $\vec{X}_\beta$ and $\vec{X}_\delta$
14: $t = t + 1$
15: **end while**
16: Return $\vec{X}_\alpha$

Algorithm 1 shows the basic GWO algorithm. As in any population-based optimization algorithm, the first step of GWO is to generate a population of n wolves (i.e., candidate solutions): population $= \langle \vec{X}_1, \vec{X}_2, \ldots, \vec{X}_n \rangle$. Each candidate solution is a vector of m dimensions (i.e., decision variables): $\vec{X}_i = \langle x_1, x_2, \ldots, x_m \rangle$. The roles in the hierarchical structure of GWO ($\vec{X}_\alpha$: first best solution, $\vec{X}_\beta$: second best solution, $\vec{X}_\delta$: third best solution and $\vec{X}\,\omega$: the other solutions) are determined using the fitness function ($f(\vec{X}_i)$) of each candidate solution $\vec{X}_i$.

The candidate solutions in GWO go through an improvement loop in an attempt to search for better solutions. The loop comprises three main steps: Tracking and chasing prey, pursuing and encircling prey and attacking prey. Encircling prey is modeled using two equations:

$$\vec{D} = |\vec{C}\,\vec{X}_p(t) - \vec{X}(t)| \tag{1}$$

$$\vec{X}(t+1) = \vec{X}_p(t) - \vec{A}\,.\,\vec{D}, \tag{2}$$

where t is the index number of the current iteration, $t + 1$ is the index number of the next iteration, $\vec{A}$ and $\vec{C}$ are two coefficient vectors, $\vec{X}_p(t)$ represents a prey (i.e., a candidate solution) and $\vec{X}(t)$ represents a wolf (i.e., a candidate solution).

The vectors $\vec{A}$ and $\vec{C}$ are calculated and updated using the following equations:

$$\vec{A} = 2\vec{a}\,.\,\vec{r}_1 - \vec{a} \tag{3}$$

$$\vec{C} = 2\vec{r}_2 \tag{4}$$

The numbers in the vector $\vec{a}$ linearly decrease from 2 to 0 with each iteration of the improvement loop of GWO. $\vec{r}_1$ and $\vec{r}_2$ are two vectors that contain random numbers between 0 and 1.

The ω candidate solutions are updated in GWO using the best three solutions in the population of possible solutions ($\vec{X}_\alpha$, $\vec{X}_\beta$ and $\vec{X}_\delta$). The following 7 equations represent the update process of the ω solutions:

$$\vec{D}_\alpha = |\vec{C}_1\vec{X}_\alpha - \vec{X}| \tag{5}$$

$$\vec{D}_\beta = |\vec{C}_2\vec{X}_\beta - \vec{X}| \tag{6}$$

$$\vec{D}_\delta = |\vec{C}_3\vec{X}_\delta - \vec{X}| \tag{7}$$

$$\vec{X}_1 = \vec{X}_\alpha - \vec{A}_1.(\vec{D}_\alpha) \tag{8}$$

$$\vec{X}_2 = \vec{X}_\beta - \vec{A}_2.(\vec{D}_\beta) \tag{9}$$

$$\vec{X}_3 = \vec{X}_\delta - \vec{A}_3.(\vec{D}_\delta) \tag{10}$$

$$\vec{X}(t+1) = \frac{\vec{X}_1 + \vec{X}_2 + \vec{X}_3}{3} \tag{11}$$

The exploitation stage (i.e., approaching and attacking prey: $\vec{A} < 1$) in GWO is modeled by linearly decreasing the values of the vector $\vec{a}$ with each iteration of the improvement loop. These values are initialized to 2 at the beginning of the improvement loop and should reach 0 at the last iteration of the loop. It is important to note that the values of vector $\vec{A}$ in Eq. (3) are in the range $[-\vec{2a}, \vec{2a}]$. The exploration stage in GWO is triggered based on two factors. First, the ω solutions change their directions away from the best solutions ($\vec{X}_\alpha$, $\vec{X}_\beta$ and $\vec{X}_\delta$), when $\vec{A} > 1$. Another important factor is the components of the vector $\vec{C}$. These values are in the range [0, 2] and act as scaling factors for the best candidate solutions in Eq. (1) by increasing their weights when $|\vec{C} > 1|$ and decreasing their weights when $|\vec{C} < 1|$.

3.2. Distributed Grey Wolf Optimizer

The Distributed Grey Wolf Optimizer (DGWO) is a distributed version of the GWO algorithm [18]. In DGWO, the population of candidate solutions is separated and organized into islands (i.e., small independent subpopulations that are organized based the island model). DGWO applies a copy of GWO to each island. The islands are only allowed to communicate with each other after a predefined period called the migration frequency (M_f). The M_f period determines the number of iterations of GWO that should be performed on each island before allowing the islands to communicate with each other. The number of candidate solutions that can be exchanged between islands is controlled by predefined percentage called the migration rate (M_r). Note that the communication process in DGWO is based on the random ring topology and the best–worst migration policy. In the random ring topology, the islands are arranged in a ring shape, where each island has a unidirectional relationship with each neighbor. Note that the arrangement of islands in the random ring topology changes each time the communication process takes place between islands. In the best–worst policy, the best m nominated solutions in one island are interchanged with the m worst nominated solutions in a neighboring island.

The DGWO has two main advantages compared to GWO. First, the island model in DGWO provides an additional exploration mechanism compared to GWO. The island model increases the chances of low-quality candidate solutions in each island to evolve into better solutions [16,17]. Second, the computational complexity of DGWO can be significantly reduced compared to the computational complexity of GWO because DGWO can be executed in parallel on multiple processing devices [18].

Algorithm 2 Distributed Grey Wolf Optimizer (DGWO).

1: Determine the total number of candidate solutions of all islands (n) and the maximum number of iterations ($MaxItr$).
2: Initialize the parameters of the island model (number of islands(s), migration frequency (M_f) and migration rate (M_r)).
3: Calculate the population size for each island ($k = n/s$)
4: Calculate the number of migration waves ($M_w = MaxItr/M_f$)
5: Calculate the number of migrant solutions ($n_r = \frac{n}{s} \times M_r$)
6: Initialize the population of k candidate solutions for each island $\overrightarrow{\boldsymbol{X}}_i^j (i = 1, 2, , k)$ and $(j = 1, 2, , s)$.
7: **for** $i = 1$ to M_w **do**
8: **for** $j = 1$ to s **do**
9: Initialize $\overrightarrow{\boldsymbol{A}}^j$, $\overrightarrow{\boldsymbol{a}}^j$ and $\overrightarrow{\boldsymbol{C}}^j$.
10: $t = 0$
11: Calculate the fitness value of each candidate solution $f(\overrightarrow{\boldsymbol{X}}_i^j)$
12: $\overrightarrow{\boldsymbol{X}}_\alpha^j$= the best candidate solution in island j
13: $\overrightarrow{\boldsymbol{X}}_\beta^j$= the second best candidate solution in island j
14: $\overrightarrow{\boldsymbol{X}}_\delta^j$= the third best candidate solution in island j
15: **while** $t < M_f$ **do**
16: **for** each candidate solution in island j **do**
17: Update the values of the current candidate solution using Eq. (11).
18: **end for**
19: Update $\overrightarrow{\boldsymbol{A}}^j$, $\overrightarrow{\boldsymbol{a}}^j$ and $\overrightarrow{\boldsymbol{C}}^j$.
20: Calculate the fitness value of each candidate solution
21: Update $\overrightarrow{\boldsymbol{X}}_\alpha^j$, $\overrightarrow{\boldsymbol{X}}_\beta^j$ and $\overrightarrow{\boldsymbol{X}}_\delta^j$
22: $t = t + 1$
23: **end while**
24: **end for**
25: **end for**
26: Calculate $\overrightarrow{\boldsymbol{X}}_\alpha (\overrightarrow{\boldsymbol{X}}_\alpha^j$ with the best fitness)
27: return $\overrightarrow{\boldsymbol{X}}_\alpha$

Algorithm 2 provides the algorithmic details of the DGWO algorithm. The first step of the algorithm is to initialize the maximum number of iterations ($MaxItr$) and the total number of candidate solutions (n). The next step is to determine the parameter setting of the island model: number of islands (s), M_f and M_r. The third step is to use a generation function to generate a k candidate solutions ($k = n/s$) for each island. Afterwards, the number of migration waves (M_w) and number of migrant solutions (n_r) are calculated.

DGWO applies the optimization mechanisms of the original GWO algorithm to each individual island. The migration process takes place after each M_f iterations of the DGWO algorithm, where a number of candidate solutions are migrated between each two neighboring islands. The number of migrant solutions between each two neighboring islands is calculated based on the migration rate (M_r). The migration process is based on a migration topology (e.g., random-ring topology) and a migration policy (e.g., best–worst migration policy). The random-ring topology used in DGWO is a dynamic topology meaning that the neighboring relationships are unidirectional relationships that automatically change before the beginning of each migration wave.

As was explained before, the migration process occurs each time M_f iterations are performed in each island $\{M_f, 2M_f, \ldots, MaxItr\}$. The first step in the migration process is to form a unidirectional ring structure based on the main principle of the random ring topology (i.e., arrange the islands randomly in a ring shape and then randomly choose an island and connect it to one of its neighbors. Keep creating unidirectional relationships in one direction until a ring is formed between all islands). The next step is applied to each island: choose the best m candidate solutions in an island and then allow these solutions to be swapped with the m worst solutions in the neighboring island based on the best–worst migration policy. Let us assume that the migration process has been initiated between two neighboring islands I_i and I_j with populations $pop_i = \{x_1^i, x_1^i, \ldots, x_k^i\}$ and $pop_j = \{x_1^j, x_1^j, \ldots, x_k^j\}$, respectively. So, assuming that $M_r = 35\%$, $n = 200$ and $s = 10$ means the number of nominated migrant solutions is $M_r \times (n/s) = 35\% \times (200/10) = 7$. Let us also assume that nominated solutions in pop_i and pop_j are arranged in ascending order based on their fitness scores: $f(x_1^i) \leq f(x_2^i) \leq \cdots \leq f(x_k^i)$ and $f(x_1^j) \leq f(x_2^j) \leq \cdots \leq f(x_k^j)$. So, for a unidirectional edge between I_i and I_j, the first seven solutions in I_i $(x_1^i, x_2^i, x_3^i, x_4^i, x_5^i, x_6^i, x_7^i)$ are going to be swapped with the last seven solutions in I_j $(x_{k-6}^j, x_{k-5}^j, x_{k-4}^j, x_{k-3}^j, x_{k-2}^j, x_{k-1}^j, x_k^j)$.

3.3. Task scheduling problem

In this section, we describe the mathematical formulation of the task scheduling problem in cloud environments (i.e., the problem of mapping tasks to compute resources for a given workflow). The purpose of task scheduling in this paper is to minimize the execution time.

3.3.1. Representation of task scheduling problem

A typical workflow application is composed of a set of n tasks $T = \{T_1, T_2, \ldots, T_n\}$, a set of m compute resources $PC = \{PC_1, PC_2, \ldots, PC_m\}$ and a set of x storage sites $S = \{S_1, S_2, \ldots, S_x\}$. Formally, an application workflow is represented as a directed acyclic graph $G = (V, E)$, where V represents the set of tasks T (i.e., the set of vertices in the graph, where each vertex represents a task: $V = \{T_1, T_2, \ldots, T_n\}$) and E is a set of data dependencies [8]. For example, $G1 = (V_1, E_1)$, where $V_1 = \{T_1, T_2, T_3\}$ and $E_1 = \{f_{12}, f_{23}\}$ means that there is a graph with two data dependencies among the tasks. The data dependency $f_{12} = \{T_1, T_2\}$ means that T_1 produces the data consumed by T_2 while $f_{23} = \{T_2, T_3\}$ means that T_2 produces the data consumed by T_3. The following assumptions are usually made about the components of a workflow application:

- The notation c_{ij} denotes the average computation time of task $T_i \in V$ in a compute resource $PC_j \in PC$. This type of cost is assumed to be known in advance for a certain size of input data for all tasks and resources.
- The notation d_{kp} denotes the data access cost from a resource k to a resource p. This type of cost is known in advance and normally fixed for cloud computing services such as Amazon and CloudFront [8].
- The transfer cost can be defined as the cost of transferring a data unit between two resources. For simplicity, the transfer cost between any two resources (say PC_i, PC_j) is assumed to be non-negative ($d_{ij} \geq 0$) and symmetric ($d_{ij} = d_{ji}$). In addition, the transfer costs should satisfy the following inequality: $d_{ij} + d_{jk} \geq d_{ik}$.

Fig. 1 shows two examples of workflow structures. Fig. 1a shows an example of a balanced workflow while Fig. 1b illustrates an example of an imbalanced workflow. Balanced workflows are simpler than imbalanced workflows [10]. Normally, a balanced workflow is composed of tasks that require similar services, but it can process different kinds of services. On the other hand, an imbalanced workflow has more complex relationships and many parallel tasks and it processes many kinds of services.

The balanced workflow in Fig. 1a is composed of five tasks while the imbalanced workflow in Fig. 1b is composed of nine tasks. The arrows in the figures represent data dependencies between the tasks. Fig. 1c shows three compute resources $PC = \{PC_1, PC_2, PC_3\}$ connected with each other, where each compute resource is stored in an independent storage unit $S = \{S_1, S_2, S_3\}$. These resources are located in different geographic locations. The cost of computation of an application is calculated by distributing its tasks over the compute resources, then the average cost of all tasks over the compute resources are adopted in the calculations. The average is measured in dollars per hour. The cost of execution of applications on resources is adopted from Pandey et al. [8] which is provided by Cloud service providers like Amazon and GoGrid. Our target is to minimize this cost as much as possible.

3.3.2. Task scheduling problem as a minimization problem

We adopted the mathematical model of the task scheduling problem as a minimization problem from [8,10,45,46]. This section provides the details of this model.

The main objective of task scheduling problem is to distribute the tasks in the set V to the compute resources in the set PC such that the sum of computation and data transmission costs is minimized for all tasks. Eqs. (12) to (16) show how the task scheduling problem can be mathematically modeled as a minimization problem [8,10]. The execution cost of a given mapping M (i.e., $C_{exe}(M)_j$ in Eq. (12)) can be calculated by adding all the node weights (i.e., the node weight is the cost of performing a task on compute resource) for each task assigned to a resource in the mapping M. The notation $C_{tx}(M)_j$ as represented in Eq. (13) is the total access cost among tasks that are computed in a compute resource PC_j and those tasks that are not computed in PC_j in the mapping. Formally, $C_{tx}(M)_j$ is the product of e_{k_1,k_2} (i.e., edge weigh between task k_1 and task k_2) and the communication cost from $M(k1)$ (i.e, the resource where k_1 is mapped) to $M(k2)$ (i.e, the resource where k_2 is mapped). Note that $d_{M(k_1),M(k_2)}e_{k_1,k_2}$ in Eq. (13) represents the average communication cost of a data unit between two resources. The cost of communication is important when $e_{k_1,k_2} > 0$, which indicates that task k_1 and task k_2 have file dependency between each other. This also means that task k_1 and task k_2 are executing in the same compute resource when $e_{k_1,k_2} = 0$.

$$C_{exe}(M)_j = \sum_{k} w_{kj} \quad \forall M(K) = j \tag{12}$$

$$C_{tx}(M)_j = \sum_{k_1 \in t} \sum_{k_2 \in t} d_{M(k_1),M(k_2)} e_{k_1,k_2} \quad \forall M(K_1) = j \text{ and } M(K_2) \neq j \tag{13}$$

$$C_{total}(M)_j = C_{exe}(M)_j + C_{tx}(M)_j \tag{14}$$

$$Cost(M) = max(C_{total}(M)_j) \quad \forall j \in P \tag{15}$$

$$Minimize(Cost(M) \quad \forall M) \tag{16}$$

Eq. (14) shows that the total cost of a given mapping M_j for compute resource PC_j is the summation of execution and access costs of M_j. Eq. (15) shows that the cost of mapping M ($Cost(M)$) is the maximum $C_{total}(M)_j$ for all $j \in P$. Eq. (16) shows that the largest cost for all the resources is minimized when evaluating the total cost for all the resources. This equation guarantees that the tasks are not mapped to a single compute resource and that the tasks are efficiently distributed to compute resources [8,10].

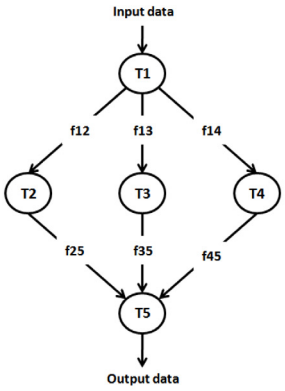
4. Distributed Grey Wolf Optimizer for cloud workflow scheduling

One of the most important services of cloud environments is the distributed computing service that provides various distributed compute resources to execute applications such as workflow applications. The scheduling process of tasks in workflow applications requires assigning the tasks to compute resources in the cloud. This process mainly depends on the availability of compute resources and the network load. Unfortunately, the scheduling process is classified as an $\mathcal{NP}$-hard problem [47]. This is because the scheduling process of tasks increases in complexity with the increase of the number of dependent tasks, number of compute resources and number of objectives (e.g., computation cost, data transmission cost). The Distributed Grey Wolf Optimizer (DGWO) is a parallel version of the Grey Wolf Optimizer (GWO) algorithm. An interesting feature of DGWO is that it can be executed in parallel machines and it has been experimentally proved to outperform many well-known optimization algorithms [18]. In the current section, we introduce a variation of DGWO for scheduling dependent tasks to compute resources based on two main objectives: Reducing both computation and data transmission costs.

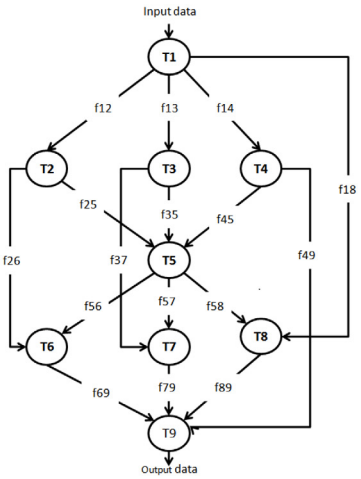
Algorithm 3 Scheduling Algorithm Based on DGWO.

1: **for** all tasks in all compute resources **do**
2: Calculate average computation cost.
3: **end for**
4: **for** each two connected compute resources **do**
5: Calculate the average communication cost between resources (communication/size of data)
6: **end for**
7: **for** each task k assigned to compute resource PC_j **do**
8: Set task node weight w_{kj} as average computation cost
9: **end for**
10: **for** each two dependent tasks (Say task k_1 and task k_2) **do**
11: Assign the edge weight e_{k_1,k_2} as the size of file transferred between tasks.
12: **end for**
13: Apply DGWO to t_i by calling DGWO in Algorithm 4 (i.e., a set of all tasks $i \in k^*$).
14: **while** there are unscheduled tasks **do**
15: **for** all ready tasks $t_i \in T$ **do**
16: Assign tasks t_i to resources p_j based on the solutions provided by DGWO
17: **end for**
18: Dispatch all the assigned tasks.
19: Wait for polling time.
20: Update the ready task list.
21: Recalculate the average cost of communication among resources based on the current networkload.
22: Compute DGWO(t_i) by calling DGWO in Algorithm 4.
23: **end while**

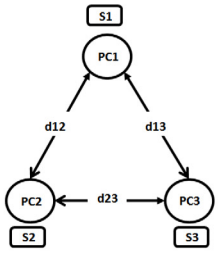
Algorithm 3 shows a new scheduling algorithm based on DGWO. This algorithm is similar to the scheduling algorithm



(a) A Balanced Workflow



(b) An Imbalanced Workflow



(c) Compute Resources

Fig. 1. Balanced/Imbalanced workflows, compute resources and storage sites.

proposed in [8], but the new algorithm uses DGWO to provide efficient solutions for workflow scheduling problems in cloud environment. The main objective of the algorithm is to produce a mapping of all tasks in a workflow to the available compute resources based on the mathematical model described in details in Section 3.3. The first step of the algorithm is to calculate the average computation cost of each task in each available compute resource. The output of the former step is the TP-matrix that contains $|T \times PC|$ entries (e.g., Table 2a). The second step is to compute the average value of communication cost between each two connected resources per unit data. The results are stored in the PP-matrix that contains $|PC \times PC|$ entries (e.g., Table 2b). Two pieces of information are assumed to be predefined in the algorithm. First, the edge weight e_{k_1,k_2} (i.e., the size of file transferred between two tasks k_1 and k_2) between each two independent tasks in the workflow. Second, the average computation cost of each task in each possible compute resources (i.e., the average computation cost of task T_k in compute resource PC_j which is denoted as w_{kj}).

Before the beginning of the scheduling process, the DGWO algorithm is used to calculate the mapping of all tasks in the workflow regardless of the task dependencies among them (Line 13). Lines 14 to 23 represent the scheduling process which is a sequence of repetitive steps that ends when all of the tasks in the workflow are scheduled. The goal here is to minimize the overall cost of mapping the tasks to available compute resources. The scheduling algorithm distributes "ready" tasks (i.e., tasks that have ready input data for execution) to compute resources based on the mapping provided by DGWO (Lines 15 to 17). The tasks are then dispatched to resources for execution. The algorithm at this point must wait for polling time (i.e., the time required to define the status of tasks) to update the list of "ready" tasks. The update process depends on the tasks that have finished execution and the data produced by each task. In Line 21, the average communication costs between resources are updated based on the status of network load. This means that DGWO must be called again (Line 22) to recalculate the mapping of tasks based on any possible change in the communication costs.

T_1	T_2	T_3	T_4	T_5
PC_1	PC_2	PC_2	PC_3	PC_1

Fig. 2. Candidate solution for the workflow in Fig. 1a.

Algorithm 4 shows the proposed variation of DGWO for task scheduling problems. In the beginning, the algorithm randomly generates n candidate solutions. The dimension (i.e, number of decision variables) of any candidate solution equals to the number of tasks in the "ready" list ($t_i \in T$). The values of decision variables in a candidate solution are the indices of the available resources. These values are randomly selected from the indices of compute resources in list PC such that not all of the tasks are executed in one compute resource. Hence, a candidate solution represents mapping of tasks to available resources (e.g., Fig. 2). The evaluation of candidate solutions is carried out as described in Section 3.2 using Eq. (16) as a fitness function. Note that the optimization process (Lines 9 to 27) of DGWO is performed for a number of times as specified by the user ($MaxItr$).

4.1. Discretization strategy of candidate solutions

The update process of the candidate solutions in each island (Lines 18–20 in Algorithm 2) produces continuous values that should be converted to discrete values. Therefore, we use the largest order value (LOV) method [25] in DGWO to determine the correct values of the decision variables in the candidate solutions.

Algorithm 4 Distributed Grey Wolf Optimizer for Task Scheduling Problem.

Set the dimension (i.e., number of decision variables) of a candidate solution as equal to the size of ready tasks $t_i \in T$.
Determine the total number of candidate solutions of all islands (n) and the maximum number of iterations ($MaxItr$).
Initialize the candidate solutions randomly from $PC = \{PC_1, PC_2, ..., PC_m\}$
Initialize the parameters of the island model (number of islands(s), migration frequency (M_f) and migration rate (M_r)).
Calculate the population size for each island ($k = n/s$)
Calculate the number of migration waves($M_w = MaxItr/M_f$)
Calculate the number of migrant solutions ($n_r = \frac{n}{s} \times M_r$)
Initialize the population of k candidate solutions for each island $\overrightarrow{\boldsymbol{X}}_i^j (i = 1, 2, , k)$ and $(j = 1, 2, , s)$.
for $i = 1$ to M_w **do**
  for $j = 1$ to s **do**
    Initialize $\overrightarrow{\boldsymbol{A}}^j$, $\overrightarrow{\boldsymbol{a}}^j$ and $\overrightarrow{\boldsymbol{C}}^j$.
    $t = 0$
    Calculate the fitness value of each candidate solution $f(\overrightarrow{\boldsymbol{X}}_i)$
    $\overrightarrow{\boldsymbol{X}}_\alpha^j$= the best candidate solution in island j
    $\overrightarrow{\boldsymbol{X}}_\beta^j$= the second best candidate solution in island j
    $\overrightarrow{\boldsymbol{X}}_\delta^j$= the third best candidate solution in island j
    while $t < M_f$ **do**
      for each candidate solution in island j **do**
        Update the values of the current candidate solution using Eq. (11).
      end for
      Update $\overrightarrow{\boldsymbol{A}}^j$, $\overrightarrow{\boldsymbol{a}}^j$ and $\overrightarrow{\boldsymbol{C}}^j$.
      Calculate the fitness value of each candidate solution
      Update $\overrightarrow{\boldsymbol{X}}_\alpha^j$, $\overrightarrow{\boldsymbol{X}}_\beta^j$ and $\overrightarrow{\boldsymbol{X}}_\delta^j$
      $t = t + 1$
    end while
  end for
end for
Calculate $\overrightarrow{\boldsymbol{X}}_\alpha$(i.e., $\overrightarrow{\boldsymbol{X}}_\alpha^j$ with the best fitness)
return $\overrightarrow{\boldsymbol{X}}_\alpha$

Table 1
Example of LOV.

A sample solution using the LOV method				
Dimension k	1	2	3	4
$x_{i,k}$	0.7	0.4	1.2	0.3
$\varphi_{i,k}$	2	3	1	4
$\pi_{i,k}$	3	1	2	4

Let the ith candidate solution $\overrightarrow{\boldsymbol{X_i}} = \langle x_{i1}, x_{i2}, \ldots, x_{in} \rangle$ be the candidate solution that contains continuous values. In LOV, $\overrightarrow{X_i}$ is used to generate a middle sequence $\overrightarrow{\boldsymbol{\varphi_i}} = \langle \varphi_{i1}, \varphi_{i2}, \ldots, \varphi_{in} \rangle$ by ordering the values of decision variables in non-decreasing order. Afterwards, the process sequence of the jobs is generated using the formula $\pi_{i,\varphi_i,k} = k(k = 1, \ldots, n)$.

In Table 1, the candidate solution is $\overrightarrow{\boldsymbol{X_i}} = \langle 0.7, 0.4, 1.2, 0.3 \rangle$. In $\overrightarrow{\boldsymbol{X_i}}$, $x_{i3} = 1.2$ is the largest number so it was given the number 1, $x_{i1} = 0.7$ is the second largest number so it was given the number 2 and so on. As a result, the middle sequence is $\overrightarrow{\boldsymbol{\varphi_i}} = \langle 0.7, 0.4, 1.2, 0.3 \rangle$. $\varphi_{i,1} = 2$, when $k = 1$, that is, $\pi_{i,\varphi_i,1} = \pi_{i,2} = 1$. $\varphi_{i,2} = 3$, when $k = 2$, that is, $\pi_{i,\varphi_i,2} = \pi_{i,3} = 2$ and so on. The final solution is $\overrightarrow{\boldsymbol{\pi_i}} = \langle 3, 1, 2, 4 \rangle$.

4.2. DGWO: A monolithic centralized scheduler

A monolithic centralized scheduler employs a single scheduler in one machine to assign tasks to VMs and to handle all workloads (e.g., JobTracker in Hadoop v1). However, it is tricky to deal with different types of applications and workloads using centralized schedulers. This is mainly because different applications have different requirements and constraints, and the performance of a centralized scheduler may degrade because of heavy incoming workloads. DGWO is a monolithic centralized approach for scheduling that is specifically designed to deal with workflow applications, and it can be used with other schedulers in the same machine. Besides, the computational complexity of DGWO can be significantly reduced compared to the computational complexity of other optimization-based scheduling algorithms because it can be executed in parallel on multiple processing devices, or in one parallel processor [18]. A parallel scheduler uses multiple schedulers in one machine based on two configurations. First, each scheduler works independently and there is no need for coordination and communications between schedulers. Second, schedulers communicate and coordinate with each other schedulers. Parallel schedulers are faster and can handle heavier workloads than centralized schedulers. However, centralized schedulers are simpler and can work besides other schedulers in the same machines [3,44,48]. We test DGWO in Section 5 using simulated data (Sections 5.3 and 5.4) and using real data and scientific workflows (Section 5.7). We also test the running time of the DGWO algorithm and other state-of-the-art algorithms (Section 5.6).

5. Experiments

This section is divided into seven subsections as follows: Section 5.1 provides the measurement of comparison, Section 5.2 describes the experimental setup for the algorithms; including data and implementation, Section 5.3 provides a comparison between the algorithms when applied to balanced workflows, Section 5.4 provides a comparison between algorithms when applied to imbalanced workflows, Section 5.5 discusses the overall experimental results and the limitations of DGWO, Section 5.6 provides a comparison of the running times of the tested algorithms, and finally Section 5.7 discusses the experimental results of DGWO, PSO and BPSO using WorkflowSim.

5.1. Performance measures

We adopted the total cost of execution for an application to measure the performance of the algorithms. We calculated the cost of execution of a given workflow using three optimization-based scheduling algorithms: DGWO, GWO [26] and PSO [8]. The calculations of the execution cost depend on the minimum completion time of a workflow over a compute resource with the maximum total cost.

5.2. Experimental setup

The experiments were conducted using 2.0 GHz quad-core 10th-generation Intel Core i5 processor with 16-GB RAM running macOS 10.15 Catalina. All of the algorithms were implemented using the Java programming language.

For balanced workflow, we have utilized four matrices that contain the following information: Table 2a shows the average

Table 2
The TP-matrix, PP-matrix and DS-matrix of balanced workflow.

(a) TP5 [5 × 3]: Cost of execution of T_i at PC_j

	PC1	PC2	PC3
T1	1.23	1.12	1.15
T2	1.17	1.17	1.28
T3	1.13	1.11	1.11
T4	1.26	1.12	1.14
T5	1.19	1.14	1.22

(b) PP [3 × 3]: Cost of communication between PC_i and PC_j

	PC1	PC2	PC3
PC1	0	0.17	0.21
PC2	0.17	0	0.22
PC3	0.21	0.22	0

(c) $DS_{T2,T3,T4}$ [2 × 2]

	Total data
i/p	10
o/p	10

(d) DS_{T5}[2 × 2]

	Total data
i/p	30
o/p	60

Table 3
The TP-matrix, PP-matrix and DS-matrix of imbalanced workflow.

(a) TP5 [9 × 3]: Cost of execution of T_i at PC_j

	PC1	PC2	PC3
T1	1.23	1.12	1.15
T2	1.17	1.17	1.28
T3	1.13	1.11	1.11
T4	1.26	1.12	1.14
T5	1.19	1.14	1.22
T6	1.17	1.17	1.28
T7	1.13	1.11	1.11
T8	1.26	1.12	1.14
T9	1.19	1.14	1.22

(b) PP [3 × 3]: Cost of communication between PC_i and PC_j

	PC1	PC2	PC3
PC1	0	0.17	0.21
PC2	0.17	0	0.22
PC3	0.21	0.22	0

(c) $DS_{T2,T3,T4}$ [2 × 2]

	Total data
i/p	10
o/p	10

(d) DS_{T5}[2 × 2]

	Total data
i/p	30
o/p	60

(e) $DS_{T6,T7,T8}$ [2 × 2]

	Total data
i/p	70
o/p	70

(f) DS_{T9}[2 × 2]

	Total data
i/p	220
o/p	440

computation cost of each task on each resource (TP-matrix). Table 2b shows the average communication cost of data transferred between compute resources (PP-matrix). Table 2c and d show the input/output Data Size of each task (DS-matrix). Note that these tables were adopted from [8].

On the other hand, for imbalanced workflow, we have used six matrices that contain the following information: Table 3a shows the average computation cost of each task on each resource (TP-matrix). Table 3b shows the average communication cost of data transferred between compute resources (PP-matrix). Table 3c, d, e and f show the input/output Data Size of each task (DS-matrix).

The TP-matrix contains different values that depend on the type of the workflow (balanced/imbalanced). For balanced workflow, we have five tasks distributed among three compute resources. On the other side, for imbalanced workflow, we have nine tasks distributed between three compute resources. The price policy in TP-matrix is based on the pricing policy of Amazon EC2 [8]. The values inside the TP-matrix are expressed in dollars per hour.

The PP-matrix contains the transfer costs between compute resources. These costs remain constants across the iterations of the algorithms, but are different in each execution. We also assumed that the compute resources are geographically dispersed with the same cost between any two resources.

There is a DS-matrix for each task. The summation of values in the DS-matrix of each task varies based on the size of data used in our experiments (64 MB, 512 MB, and 5 GB). In Figs. 1a and 1b, if T_1 produces data of size x then the data dependencies f_{12}, f_{13} and f_{14} indicate that each of T_2, T_3 and T_4 consumes the x data produced by T_1. In addition, the data dependencies f_{25}, f_{35} and f_{45} indicate that T_5 consumes the $3x$ data produced by T_2, T_3 and T_4. The task T_5 produces $6x$ data. In the algorithms that we have implemented (DGWQ, GWO and PSO), the numbers of candidate solutions are 5 and 25. The number of iterations used to obtain a solution is 45 and the number of independent executions is 30 as recommended in [8].

The values of the parameters of the optimization algorithms were as follows:

- The weight parameters of PSO were C1=1, C2=1 and W=0. The values of these parameters were finely tuned based on extensive simulations.
- In DGWO, the parameters of the island model were s = 10, M_f = 5 and M_r = 10% as recommended in [17,18].
- The numbers in the vector $\vec{a}$ were designed to linearly decrease from 2 to 0 with each iteration of the improvement loops in GWO and DGWO as described in [18,26].

The source code of DGWO is available at: https://www.dropbox.com/s/2d16t46598u03y0/DistributedGreyWolfOptimizer.zip?dl=0

5.3. Experimental results of balanced workflows

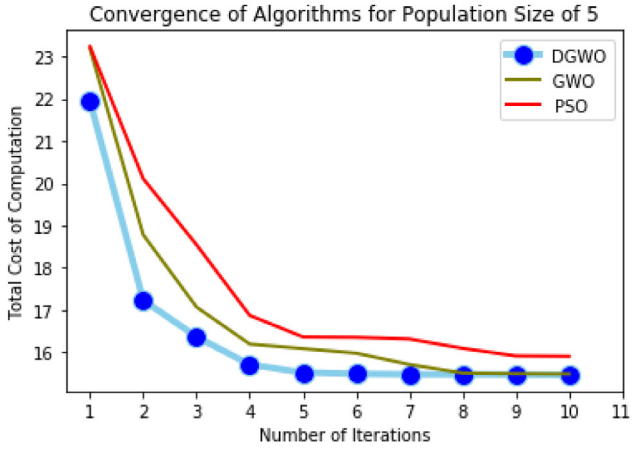
We evaluated the performance of the optimization-based scheduling algorithms using the balanced workflow in Fig. 1a. The tasks in the workflow consume and produce data of different sizes. The population can vary from small to big sizes. In addition, the execution cost of a task is different from one computer resource to another as in Table 2a. We show the performance of DGWO through changing these factors in turns.

We applied the algorithms for 45 iterations, but we show the first 10 iterations. Figs. 3a, 3b and 3c show the convergence of the average total cost for DGWO in comparison to GWO and PSO for a population size of 5. It is computed across the number of iterations for different data sizes (64 MB, 512 MB and 5 GB). Figs. 4a, 4b and 4c display the convergence of the algorithms for a population size of 25. The difference in average total cost is significant when the population size increases.

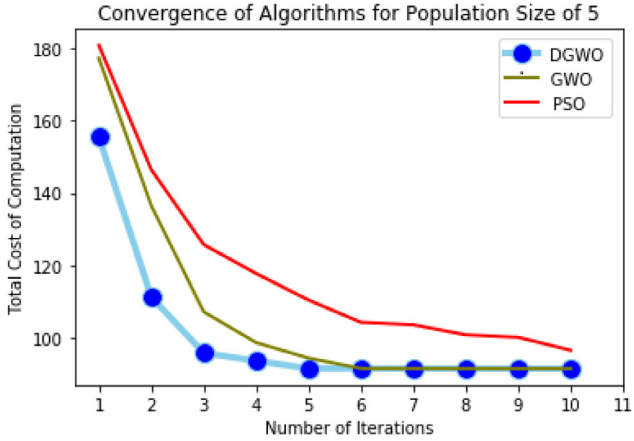
It is found that the convergence rate is relative to the population size. As in Fig. 4, when the population size is 25, DGWO converges from the second iteration, while it converges after 5 iterations when the size of population is 5 as in Fig. 3. This indicates that the increase of population size gives a clear picture of the efficiency of DGWO compared to other algorithms.

From another point of view, when the population size is 25, PSO converges after 6 iterations as in Figs. 4a and 4b which show the convergence for smaller data sizes 64 MB and 512 MB respectively. When the data size was increased to 5 GB (Fig. 4c), PSO shows minor enhancement (It converges starting from the fourth iteration) compared to smaller data sizes. But, when compared to DGWO, PSO converges slower in all the cases. It is also clear from Fig. 3 that PSO fails to converge even for the first 10 iterations when the population size is 5.

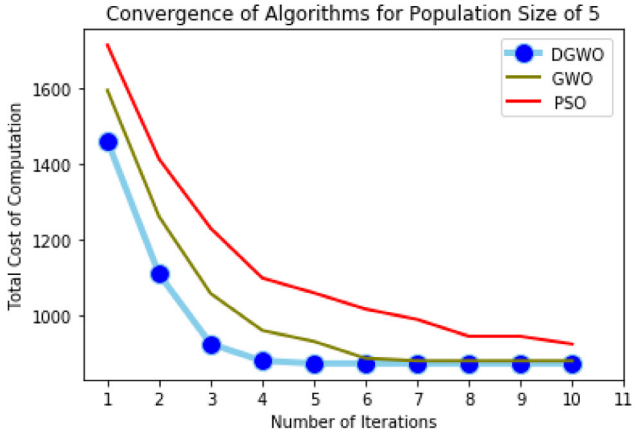
For the case of GWO, it is categorized in an intermediate situation between PSO and DGWO, particularly it is much closer to PSO than DGWO. This means that DGWO is the dominant algorithm over the others. By referring to Fig. 3, GWO converges to solutions after the sixth iteration for large data sizes (512 MB and 5 GB), and after 8 iterations for small data size (64 MB), but again, it is better than PSO, since PSO never converge in the first 10 iterations as discussed before. The same scenario occurs when the population size is 25, and explicitly, in the case of the largest data size, it is noticeable that both GWO and PSO converge to



(a) Convergence of Algorithms when Data Size is 64 MB.

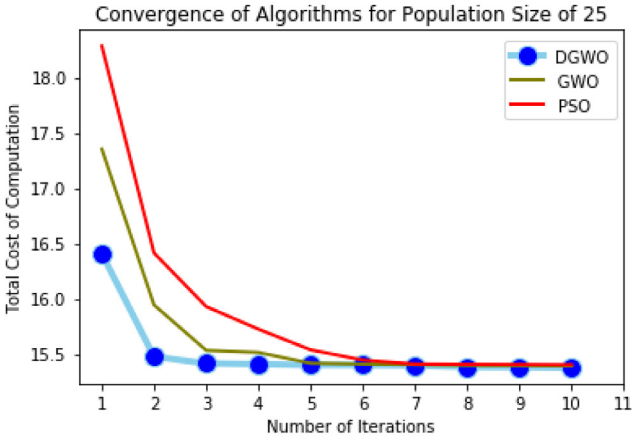


(b) Convergence of Algorithms when Data Size is 512 MB.

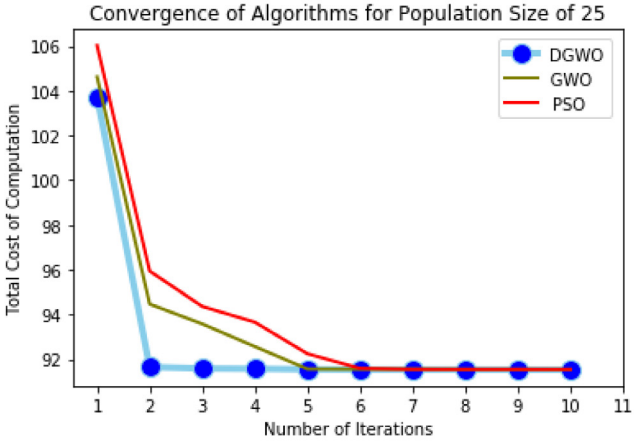


(c) Convergence of Algorithms when Data Size is 5 GB.

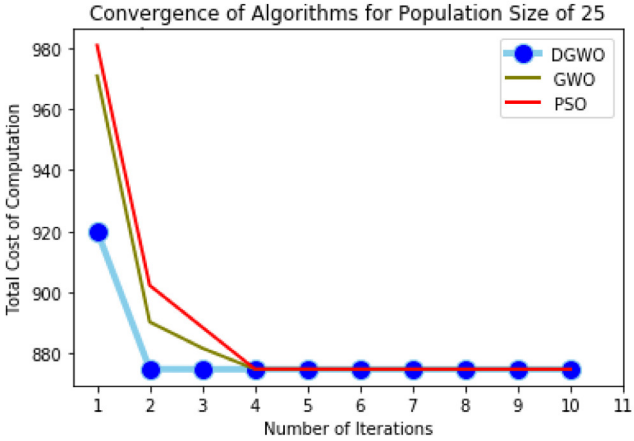
Fig. 3. Convergence of algorithms for population size of 5 in balanced workflows.



(a) Convergence of Algorithms when Data Size is 64 MB.



(b) Convergence of Algorithms when Data Size is 512 MB.



(c) Convergence of Algorithms when Data Size is 5 GB.

Fig. 4. Convergence of algorithms for population size of 25 in balanced workflows.

solutions after the fourth iteration. But, still DGWO shows the best case of convergence to solutions.

The overall experimental results on balanced workflows indicate that PSO produces the worst results compared to the other algorithms, specially, for small population sizes. In contrast, DGWO outperforms both GWO and PSO when applied to different data sizes. Obviously, in the early iterations, DGWO discovers a global minima faster than other algorithms.

5.4. Experimental results of imbalanced workflows

We evaluated the performance of task scheduling problem when dealing with imbalanced workflows, such that loads are balanced by taking into account data dependencies across multiple levels as depicted in Fig. 1b. We adopted the execution costs shown in Table 3a. Here, we discussed the performance of DGWO

and other algorithms by changing two factors (the data size and population size) alternately.

In similar to balanced workflows, we tested the behavior of the algorithms across 45 iterations, but we display the first 10 iterations. Figs. 5 and 6 present the convergence of DGWO compared to GWO and PSO in terms of the average total cost. We applied the algorithms for two population sizes (5 and 25) over all iterations for different data sizes (64 MB, 512 MB and 5 GB).

Fig. 5 demonstrates the convergence of the algorithms when the population size is 5. Regardless of the data size, the average total cost of execution decreases rapidly for DGWO, where it approaches solutions faster than PSO and GWO. When applied to two data sizes of 512 MB and 5 GB (Figs. 5b and 5c), the difference of convergence rate seems obvious in favor of DGWO compared to GWO, precisely, DGWO gets closer to solutions starting from the second iteration. In contrast, PSO exhibits poor performance for different data sizes and never converge, especially when applied to small data size of 64 MB (Fig. 5a).

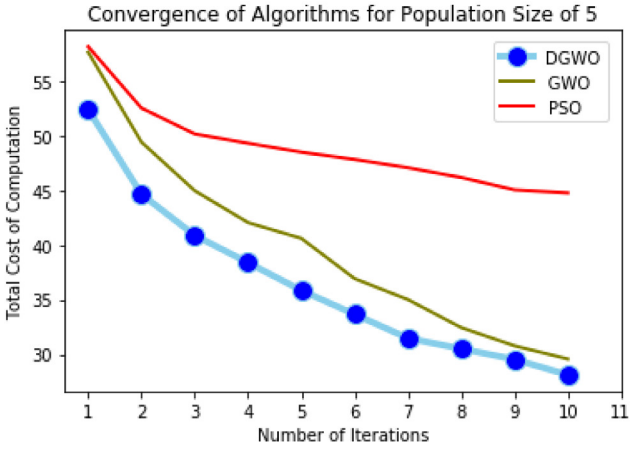
Fig. 6 shows the convergence of algorithms when the population size increases to 25. Figs. 6b and 6c show the convergence of algorithms when the data sizes are 512 MB and 5 GB respectively. For both data sizes, DGWO converges significantly to solutions starting from the sixth iteration. Even though GWO presents closer behavior to DGWO, but still DGWO achieves better results with regard to the objective function (minimization of total average cost of application execution). In addition, DGWO converges to solutions at the tenth iteration when the data size is 64 MB as shown in Fig. 6a, while GWO and PSO fail to converge during the first 10 iterations. Furthermore, PSO appears to converge slowly to solutions, the same as or slightly better than in the case of the small population size of 5.

Generally, it is noticeable that PSO has low performance compared to the other algorithms, especially when applied to problems that have small population size. On the other hand, DGWO approaches solutions faster than GWO and PSO, and it is apparently achieves better results for different data sizes as shown in Figs. 5 and 6.

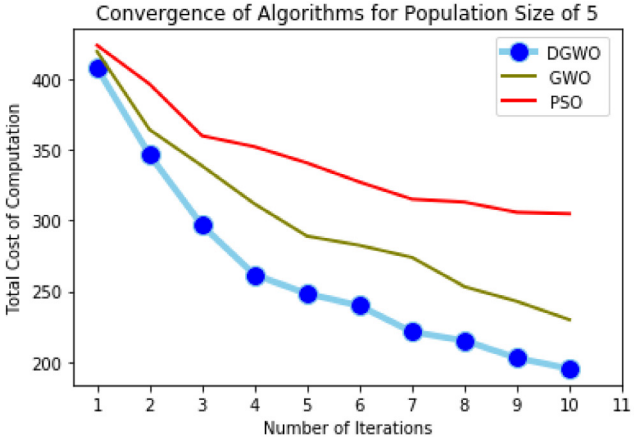
5.5. Discussion of the overall experimental results and limitations

The overall experimental results conducted on imbalanced workflows compared to the ones conducted on balanced workflows show more clearly that DGWO performs better than GWO and PSO. The results also indicate that GWO and PSO require more simulations to produce comparable results. The experimental results in all types of workflows suggest that the execution costs on compute resources rapidly increase as the number of tasks increases. From another point of view, the size of the population affects the way the algorithms converge to solutions, where it appears more clear in small sizes. In addition, when the data size of an application increases, the algorithms show big difference between them in approaching solutions. However, DGWO remains the fastest algorithm, and there is no doubt that it converges faster to global minima compared to GWO and PSO algorithms.

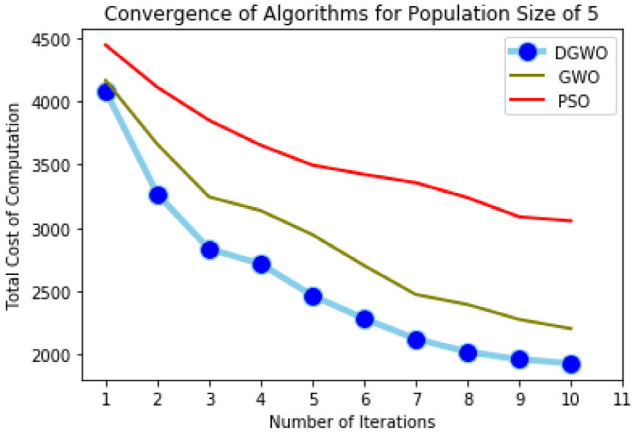
According to the complexity analysis in [18], the computational complexity of DGWO is $O(M_w.s.M_f.k)$, while the computational complexity of GWO is $O(m.n)$, where m is the maximum number of iterations and n is the number of candidate solutions. Therefore, it is better to execute DGWO in parallel machines which will reduce its computational complexity to $O((M_w.s.M_f.k)/s)$. This means that the computational complexity of DGWO is $O(M_w.M_f.k)$, which is equivalent to the computational complexity of GWO [18].



(a) Convergence of Algorithms when Data Size is 64 MB.



(b) Convergence of Algorithms when Data Size is 512 MB.

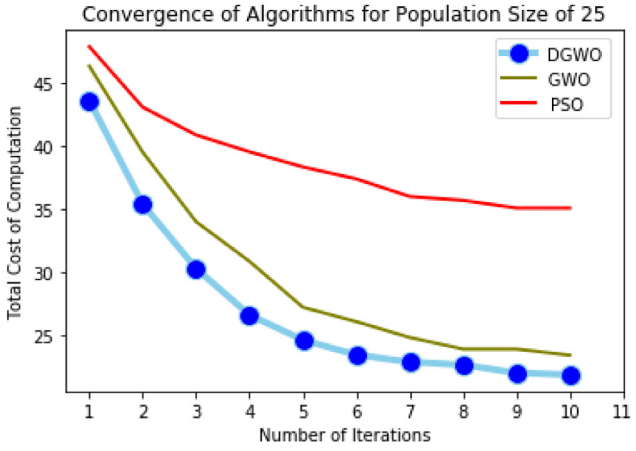


(c) Convergence of Algorithms when Data Size is 5 GB.

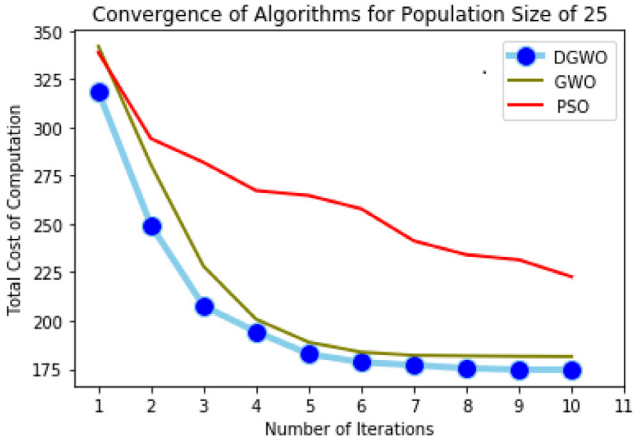
Fig. 5. Convergence of algorithms for population size of 5 in imbalanced workflows.

5.6. Comparison of running times of GWO, PSO and DGWO

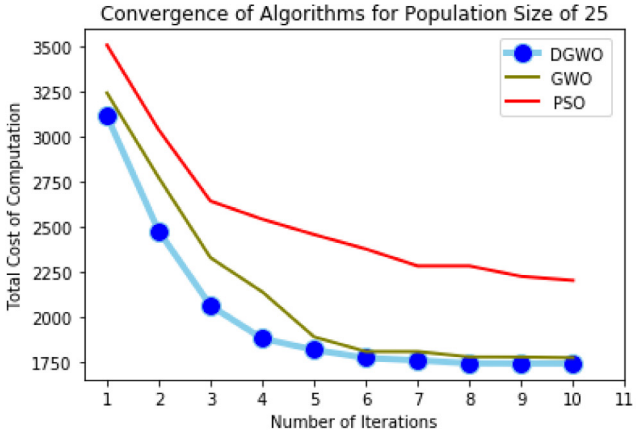
Tables 4 and 5 show the running time of GWO, PSO and DGWO when they were applied to a balanced workflow (Section 5.3), while Tables 6 and 7 show the running time of these algorithms when they were applied to an imbalanced workflow (Section 5.4). The results were reported in milliseconds representing the average of 30 independent runs. The running time of each algorithm



(a) Convergence of Algorithms when Data Size is 64 MB.



(b) Convergence of Algorithms when Data Size is 512 MB.



(c) Convergence of Algorithms when Data Size is 5 GB.

Fig. 6. Convergence of algorithms for population size of 25 in imbalanced workflows.

was reported when it converged to the total cost of computation of DGWO (Sections 5.3 and 5.4).

The size of population in Table 4 is 5, while the size of population in Table 5 is 25. While the running time of all algorithms increases as the size of data increases, DGWO is much faster for all data sizes.

The size of population in Table 6 is 5, while the size of population in Table 7 is 25. The results in the tables confirm that

Table 4
The running time of DGWO vs. the running time of the other algorithms in milliseconds. The size of population is 5 and the workflow is balanced.

	PSO	GWO	DGWO
64 MB	1811	1160	1054
512 MB	1945	1612	1191
5 GB	3031	2120	1890

Table 5
The running time of DGWO vs. the running time of the other algorithms in milliseconds. The population size is 25 and the workflow is balanced.

	PSO	GWO	DGWO
64 MB	7936	6029	5593
512 MB	9258	7890	5875
5 GB	13 127	11 970	10 992

Table 6
The running time of DGWO vs. the running time of the other algorithms in milliseconds. The size of population is 5 and the workflow is imbalanced.

	PSO	GWO	DGWO
64 MB	*	5649	4930
512 MB	*	9222	7930
5 GB	*	12 842	9725

Table 7
The running time of DGWO vs. the running time of the other algorithms in milliseconds. The population size is 25 and the workflow is imbalanced.

	PSO	GWO	DGWO
64 MB	*	7285	6383
512 MB	*	10 066	8432
5 GB	*	14 968	11 560

DGWO is the fastest algorithm for all data sizes. Note that PSO is the only algorithm that did not converge to similar solutions to DGWO, denoted by *, within 45 iterations. This is maybe because the optimization operators of PSO are not efficient when the are applied to imbalanced workflow.

However, we noticed the running time of all the algorithms increases when the size of population increases from 5 to 25. This is because the optimization operators of the algorithms have to be applied in the first case to 5 candidate solutions, while in the second case they have to be applied to 25 solutions.

5.7. *Performance analysis of DGWO, PSO and BPSO using different scientific workflows*

In this section, WorkflowSim [49] was used to evaluate the performance of DGWO compared to the performance results discussed in [27] for two well-known scheduling algorithms: BPSO (Binary PSO) and PSO. WorkflowSim is a toolkit for evaluating scientific workflows in distributed environments. Three workflow structures were used to simulate the results: Montage, Inspiral and Epigenomics [50]. The workflows were tested for four different problem sizes 25, 50, 100 and 1000. These workflows were generated using pegasus workflow generator [51]. The same properties of VMs as defined in [27] were applied to DGWO: the processing speed between VMs is between 400 and 2000 MIPS (Million Instructions Per Second), the average bandwidth between VMs is the same as in Amazon web services (1000 Mbps) and the data transfer time within a VM is zero. Further information about the parameter settings of BPSO and PSO are described in [27]. The parameter settings of DGWO were as follows: Population size = 100, Maximum iterations = 50, $s = 10$, $M_f = 5$ and $M_r = 10\%$ [17,18].

Figs. 7–9 show the average makespan over 10 runs for all the algorithms when applied to three workflows. In the figures,

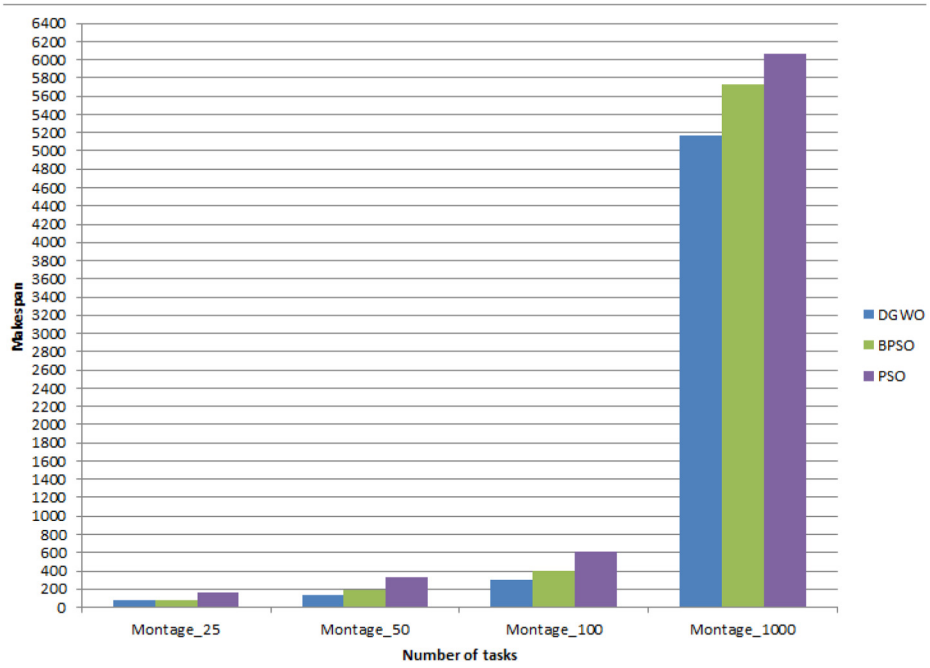


Fig. 7. Makespan of Montage workflow.

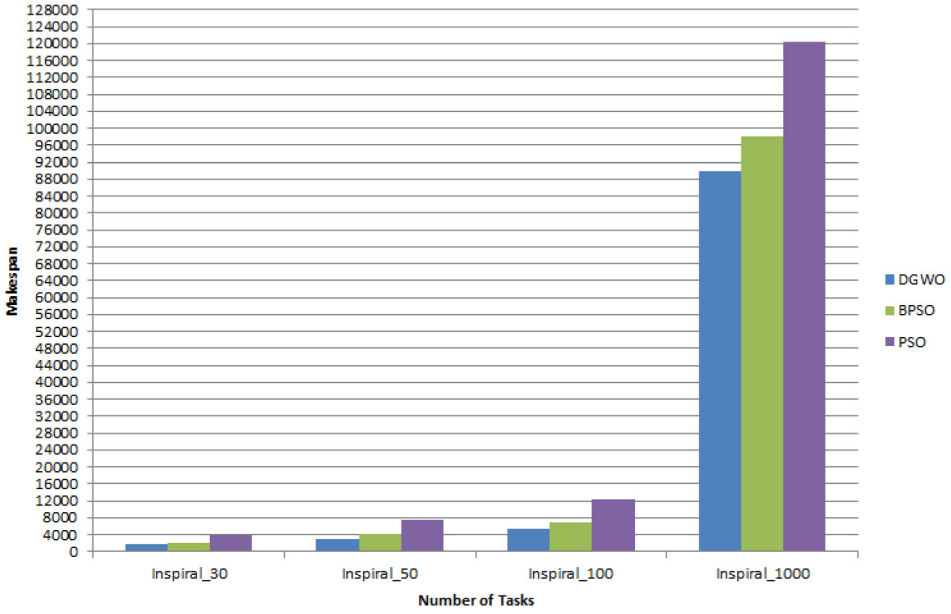


Fig. 8. Makespan of Inspiral workflow.

the number of iterations is 50 and the number of VMs is 5. We observed from the three figures that DGWO achieved the lowest makespan compared to BPSO and PSO. These results strongly indicate that the performance of DGWO improves as the number of tasks increases compared to the other algorithms. This is an evidence of the strength and efficiency of DGWO when applied to various workflows with different sizes.

6. Conclusions and future work

The current paper presented an optimization-based scheduling algorithm for workflow applications based on the Distributed Grey Wolf Optimizer (DGWO) algorithm. The task scheduling problem of workflow applications was modeled as an optimization problem as follows: Provide a mapping of dependent tasks of a workflow application to compute resources on a cloud computing environment such that the total execution cost (computation and data transmission costs) of the application is minimized.

The proposed algorithm was experimentally tested and compared to two well-known optimization-based scheduling algorithms (PSO, GWO) using two types of workflows: balanced and imbalanced workflows. In general, two conclusions can be drawn from the experimental results. First, the overall experimental results on balanced workflows suggest that DGWO performs better than GWO and PSO when applied to various data sizes. In addition, PSO has the worst performance compared to the other algorithms, especially, for small population sizes. Second, the overall experimental results conducted on imbalanced workflows compared to the ones conducted on balanced workflows indicate

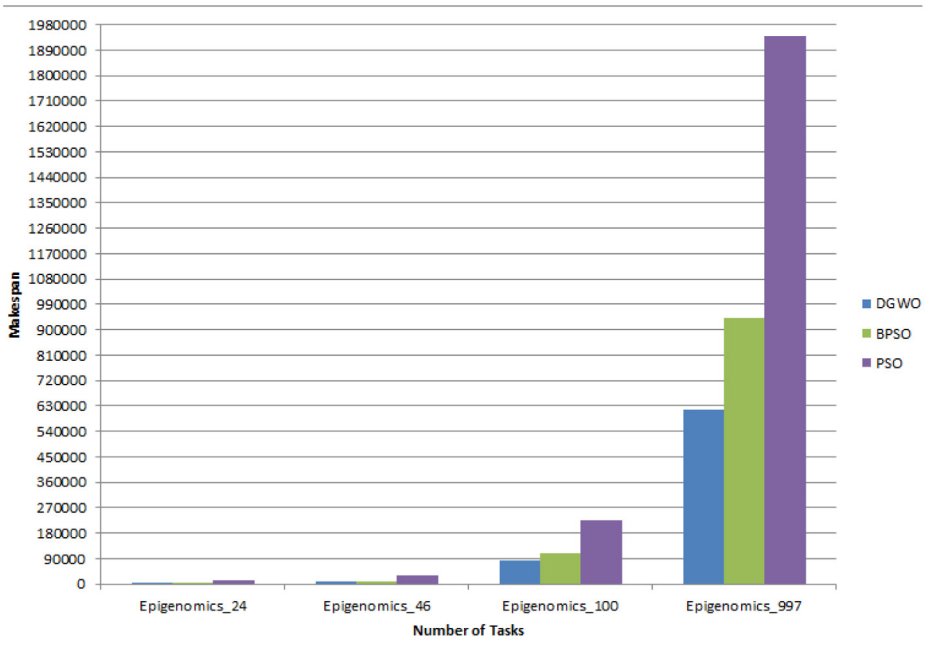


Fig. 9. Makespan of Epigenomics workflow.

more clearly that DGWO performs better than GWO and PSO. This means that GWO and PSO require more computations to produce comparable results to DGWO.

Besides, the performance of DGWO was evaluated using WorkflowSim and three scientific workflows of different sizes. The obtained simulation results compared to the reported results in [27] for two discrete variations of PSO indicate that DGWO provides the best makespan compared to the other tested algorithms.

There are four interesting directions for future work. First, the search space in the Q-learning algorithm [52,53] and the multi-agent cooperative Q-learning algorithm [54,55] can be modeled as an optimization problem, where the population of Q-values (i.e., the basic units of information in Q-learning that represent the values of state–action pairs) represents a population of candidate solutions and the Q-function represents the objective function. Inspired by the previous facts, the DGWO algorithm will be applied to the Q-learning algorithm and the multi-agent cooperative Q-learning algorithm as discussed in [56,57]. Second, DGWO would be incorporated into an intelligent distributed recommender system based on the problem model described in [58]. Third, we will attempt to solve the scheduling problem for workflow applications using other island-based optimization algorithms such as island artificial bee colony [59], island flower pollination algorithm [60], island bat algorithm [61], island-based harmony search [62] and island-based genetic algorithm [63]. Fourth, it would be interesting to evaluate the performance of DGWO using complex scientific workflow applications in cloud computing.

CRediT authorship contribution statement

Bilal H. Abed-alguni: Conceptualization, Methodology, Investigation, Validation, Writing - original draft, Supervision. **Noor Aldeen Alawad:** Experimentation, Visualization, Reviewing and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] S. Mehta, P. Kaur, Scheduling data intensive scientific workflows in cloud environment using nature inspired algorithms, in: Nature-Inspired Algorithms for Big Data Frameworks, IGI Global, 2019, pp. 196–217.
[2] D. Fernández-Cerero, A. Jakóbik, A. Fernández-Montes, J. Kołodziej, Gamescore: Game-based energy-aware cloud scheduler and simulator for computational clouds, Simul. Model. Pract. Theory 93 (2019) 3–20.
[3] M. Schwarzkopf, A. Konwinski, M. Abd-El-Malek, J. Wilkes, Omega: flexible, scalable schedulers for large compute clusters, in: Proceedings of the 8th ACM European Conference on Computer Systems, 2013, pp. 351–364.
[4] D. Fernández-Cerero, F.J. Ortega-Irizo, A. Fernández-Montes, F. Velasco-Morente, Bullfighting extreme scenarios in efficient hyper-scale cluster computing, Cluster Comput. (2020) 1–17.
[5] H. Rafat, M. Barhoush, B.H. Abed-alguni, A semantic-based approach for managing healthcare big data: A survey, J. Healthc. Eng. 2020 (2020) 1–12.
[6] H.T. Dinh, C. Lee, D. Niyato, P. Wang, A survey of mobile cloud computing: architecture, applications, and approaches, Wirel. Commun. Mob. Comput. 13 (18) (2013) 1587–1611.
[7] A. Rajagopalan, D.R. Modale, R. Senthilkumar, Optimal scheduling of tasks in cloud computing using hybrid firefly-genetic algorithm, in: Advances in Decision Sciences, Image Processing, Security and Computer Vision, Springer, 2020, pp. 678–687.
[8] S. Pandey, L. Wu, S.M. Guru, R. Buyya, A particle swarm optimization-based heuristic for scheduling workflow applications in cloud computing environments, in: 2010 24th IEEE International Conference on Advanced Information Networking and Applications, IEEE, 2010, pp. 400–407.
[9] A. Awad, N. El-Hefnawy, H. Abdel_kader, Enhanced particle swarm optimization for task scheduling in cloud computing environments, Procedia Comput. Sci. 65 (2015) 920–929.
[10] A. Khalili, S.M. Babamir, Optimal scheduling workflows in cloud computing environment using Pareto-based Grey Wolf Optimizer, Concurr. Comput.: Pract. Exper. 29 (11) (2017) e4044.
[11] Z.-G. Chen, Z.-H. Zhan, Y. Lin, Y.-J. Gong, T.-L. Gu, F. Zhao, H.-Q. Yuan, X. Chen, Q. Li, J. Zhang, Multiobjective cloud workflow scheduling: a multiple populations ant colony system approach, IEEE Trans. Cybern. 49 (8) (2018) 2912–2926.
[12] A. Choudhary, I. Gupta, V. Singh, P.K. Jana, A GSA based hybrid algorithm for bi-objective workflow scheduling in cloud computing, Future Gener. Comput. Syst. 83 (2018) 14–26.
[13] L. Liu, M. Zhang, R. Buyya, Q. Fan, Deadline-constrained coevolutionary genetic algorithm for scientific workflow scheduling in cloud computing, Concurr. Comput.: Pract. Exper. 29 (5) (2017) e3942.
[14] S. Raghavan, P. Sarwesh, C. Marimuthu, K. Chandrasekaran, Bat algorithm for scheduling workflow applications in cloud, in: 2015 International Conference on Electronic Design, Computer Networks & Automated Verification (EDCAV), IEEE, 2015, pp. 139–144.
[15] B.H. Abed-alguni, A.F. Klaib, Hybrid whale optimization and β-hill climbing algorithm, Int. J. Comput. Sci. Math. 0 (0) (2018) 1–13.

[16] B.H. Abed-alguni, A.F. Klaib, K.M. Nahar, Island-based whale optimization algorithm for continuous optimization problems, Int. J. Reason. Based Intell. Syst. (2019) 1–11.
[17] B.H. Abed-alguni, Island-based cuckoo search with highly disruptive polynomial mutatio, Int. J. Artif. Intell. (2019) 1–30.
[18] B.H. Abed-alguni, M. Barhoush, Distributed grey wolf optimizer for numerical optimization problems, Jordanian J. Comput. Inf. Technol. (JJCIT) 4 (03) (2018).
[19] B.H. Abed-alguni, F. Alkhateeb, Intelligent hybrid cuckoo search and β-hill climbing algorithm, J. King Saud Univ. Comput. Inf. Sci. 0 (0) (2018) 1–43.
[20] X.-S. Yang, S. Deb, Cuckoo search via Lévy flights, in: World Congress on Nature & Biologically Inspired Computing, 2009. NaBIC 2009, IEEE, 2009, pp. 210–214.
[21] F. Alkhateeb, B.H. Abed-Alguni, A hybrid cuckoo search and simulated annealing algorithm, J. Intell. Syst. (2017).
[22] B.H. Abed-alguni, F. Alkhateeb, Novel selection schemes for cuckoo search, Arab. J. Sci. Eng. 42 (8) (2017) 3635–3654.
[23] S.R. KS, S. Murugan, Memory based hybrid dragonfly algorithm for numerical optimization problems, Expert Syst. Appl. 83 (2017) 63–78.
[24] C. Yu, L. Kelley, S. Zheng, Y. Tan, Fireworks algorithm with differential mutation for solving the CEC 2014 competition problems, in: 2014 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2014, pp. 3238–3245.
[25] B. Qian, L. Wang, R. Hu, W.-L. Wang, D.-X. Huang, X. Wang, A hybrid differential evolution method for permutation flow-shop scheduling, Int. J. Adv. Manuf. Technol. 38 (7–8) (2008) 757–777.
[26] S. Mirjalili, S.M. Mirjalili, A. Lewis, Grey wolf optimizer, Adv. Eng. Softw. 69 (2014) 46–61.
[27] B. Kumar, M. Kalra, P. Singh, Discrete binary cat swarm optimization for scheduling workflow applications in cloud systems, in: 2017 3rd International Conference on Computational Intelligence & Communication Technology (CICT), IEEE, 2017, pp. 1–6.
[28] A.S. Kumar, M. Venkatesan, Task scheduling in a cloud computing environment using hgpso algorithm, Cluster Comput. (2018) 1–7.
[29] Z. Abdmouleh, A. Gastli, L. Ben-Brahim, M. Haouari, N.A. Al-Emadi, Review of optimization techniques applied for the integration of distributed generation from renewable energy sources, Renew. Energy 113 (2017) 266–280.
[30] M. Abdullahi, M.A. Ngadi, S.I. Dishing, B.I. Ahmad, et al., An efficient symbiotic organisms search algorithm with chaotic optimization strategy for multi-objective task scheduling problems in cloud computing environment, J. Netw. Comput. Appl. 133 (2019) 60–74.
[31] Z. Zhou, F. Li, H. Zhu, H. Xie, J.H. Abawajy, M.U. Chowdhury, An improved genetic algorithm using greedy strategy toward task scheduling optimization in cloud environments, Neural Comput. Appl. (2019) 1–11.
[32] M. Amoon, N. El-Bahnasawy, M. ElKazaz, An efficient cost-based algorithm for scheduling workflow tasks in cloud computing systems, Neural Comput. Appl. 31 (5) (2019) 1353–1363.
[33] M. Kumar, S. Sharma, Pso-based novel resource scheduling technique to improve qos parameters in cloud computing, Neural Comput. Appl. (2019) 1–24.
[34] K.P. Kumar, K. Kousalya, Amelioration of task scheduling in cloud computing using crow search algorithm, Neural Comput. Appl. (2019) 1–7.
[35] K.Y. Lee, J.-B. Park, Application of particle swarm optimization to economic dispatch problem: advantages and disadvantages, in: 2006 IEEE PES Power Systems Conference and Exposition, IEEE, 2006, pp. 188–192.
[36] E. Zitzler, M. Laumanns, L. Thiele, Spea2: Improving the strength Pareto evolutionary algorithm, in: TIK-Report, Vol. 103, Eidgenössische Technische Hochschule Zürich (ETH), Institut für Technische …, 2001.
[37] I. Ellabib, P. Calamai, O. Basir, Exchange strategies for multiple ant colony system, Inform. Sci. 177 (5) (2007) 1248–1264.
[38] Y. Zhang, Z. Liu, F. Yu, T. Jiang, Cloud computing resources scheduling optimisation based on improved bat algorithm via wavelet perturbations, Int. J. High Perform. Syst. Archit. 7 (4) (2017) 189–196.
[39] L. Li, Y. Zhou, A novel complex-valued bat algorithm, Neural Comput. Appl. 25 (6) (2014) 1369–1381.
[40] D. Gabi, A.S. Ismail, A. Zainal, Z. Zakaria, A. Abraham, Orthogonal taguchi-based cat algorithm for solving task scheduling problem in cloud computing, Neural Comput. Appl. 30 (6) (2018) 1845–1863.
[41] P. Krishnadoss, P. Jacob, Ocsa: task scheduling algorithm in cloud computing environment, Int. J. Intell. Eng. Syst. 11 (3) (2018) 271–279.
[42] G.G. Tejani, V.J. Savsani, V.K. Patel, Adaptive symbiotic organisms search (sos) algorithm for structural design optimization, J. Comput. Des. Eng. 3 (3) (2016) 226–249.
[43] A. Lal, C.R. Krishna, Critical path-based ant colony optimization for scientific workflow scheduling in cloud computing under deadline constraint, in: Ambient Communications and Computer Systems, Springer, 2018, pp. 447–461.
[44] B. Hindman, A. Konwinski, M. Zaharia, A. Ghodsi, A.D. Joseph, R.H. Katz, S. Shenker, I. Stoica, Mesos: A platform for fine-grained resource sharing in the data center., in: NSDI, Vol. 11, 2011, p. 22.
[45] E. Alkhanak, S. Lee, T. Ling, A hyper-heuristic approach using a prioritized selection strategy for workflow scheduling in cloud computing, in: International Conference on Computer Science and Computational Mathematics (ICCSCM), 2015, pp. 601–608.
[46] J. Yu, R. Buyya, A budget constrained scheduling of workflow applications on utility grids using genetic algorithms, in: 2006 Workshop on Workflows in Support of Large-Scale Science, IEEE, 2006, pp. 1–10.
[47] J.D. Ullman, Np-complete scheduling problems, J. Comput. Syst. Sci. 10 (3) (1975) 384–393.
[48] B. Golden, Amazon Web Services for Dummies, John Wiley & Sons, 2013.
[49] W. Chen, E. Deelman, Workflowsim: A toolkit for simulating scientific workflows in distributed environments, in: 2012 IEEE 8th International Conference on E-Science, IEEE, 2012, pp. 1–8.
[50] D. Hollingsworth, U. Hampshire, Workflow management coalition: The workflow reference model, in: Document Number TC00-1003, Vol. 19, Citeseer, 1995, p. 224, no. 16.
[51] Workflow Generator Pegasus.
[52] C. Watkins, Learning from Delayed Rewards, (PhD thesis), Cambridge University, Cambridge, England, 1989.
[53] B.H. Abed-alguni, M.A. Ottom, Double delayed Q-learning, Int. J. Artif. Intell. 16 (2) (2018) 41–59.
[54] B.H. Abed-Alguni, D.J. Paul, S.K. Chalup, F.A. Henskens, A comparison study of cooperative Q-learning algorithms for independent learners, Int. J. Artif. Intell. 14 (1) (2016) 71–93.
[55] B.H. Abed-alguni, S.K. Chalup, F.A. Henskens, D.J. Paul, A multi-agent cooperative reinforcement learning model using a hierarchy of consultants, tutors and workers, Vietnam J. Comput. Sci. 2 (4) (2015) 213–226.
[56] B.H. Abed-alguni, Action-selection method for reinforcement learning based on cuckoo search algorithm, Arab. J. Sci. Eng. 43 (12) (2018) 6771–6785.
[57] B.H. Abed-alguni, Bat q-learning algorithm, Jordanian J. Comput. Inf. Technol. (JJCIT) 3 (1) (2017) 56–77.
[58] N.A. Alawad, A. Anagnostopoulos, S. Leonardi, I. Mele, F. Silvestri, Network-aware recommendations of novel tweets, in: Proceedings of the 39th International ACM SIGIR Conference on Research and Development in Information Retrieval, ACM, 2016, pp. 913–916.
[59] M.A. Awadallah, M.A. Al-Betar, A.L. Bolaji, I.A. Doush, A.I. Hammouri, M. Mafarja, Island artificial bee colony for global optimization, Soft Comput. (2020) 1–27.
[60] M.A. Al-Betar, M.A. Awadallah, I.A. Doush, A.I. Hammouri, M. Mafarja, Z.A.A. Alyasseri, Island flower pollination algorithm for global optimization, J. Supercomput. 75 (8) (2019) 5280–5323.
[61] M.A. Al-Betar, M.A. Awadallah, Island bat algorithm for optimization, Expert Syst. Appl. 107 (2018) 126–145.
[62] M.A. Al-Betar, M.A. Awadallah, A.T. Khader, Z.A. Abdalkareem, Island-based harmony search for optimization problems, Expert Syst. Appl. 42 (4) (2015) 2026–2035.
[63] S.M.Z. Mohammed, A.T. Khader, M.A. Al-Betar, 3-sat using island-based genetic algorithm, IEEJ Trans. Electron. Inf. Syst. 136 (12) (2016) 1694–1698.